

Storage Cost

A Formal Model for Effective \$/GB and Storage Efficiency for Centera

Document Version: 1.00 (revised September 14, 2026)

Robert Primmer[†], Jiri Schindler[†], Jim Espy[†],
Carl D'Halluin[‡]

Centera Division

[†] Hopkinton, MA USA

[‡] Mechelen, Belgium

October 25, 2005

This document describes a formal model which will be used to calculate the effective \$:GB for Centera. Additionally, this model is generalized to provide storage efficiency and measure marginal effects of introducing new product ideas.

Contents

1	Models	1
2	Notation	2
2.1	Notes	3
3	Equations	4
4	Model Considerations	12
4.1	Multiple Protection Schemes	12
4.2	Database Overhead	12
5	EMC Cost Categorization	13
6	References	14
7	Acknowledgements	14

1 Models

The goal is to forecast effective cost per GB for Safari 2/Hapkin Centera configurations. The models below successively account for fixed overhead, data protection, field uplift, gross margin, and object-count limits. Each M_i is a cost or price per GB, not a capacity.

M_0 Raw material cost per GB: $M_0 = T/G_R$.

M_1 Material cost per available GB after fixed overhead: $M_1 = T/G_A$, where $G_A = G_R - \sum_{i=0}^3 H_i$.

M_2 Material cost per potentially usable GB after data protection: $M_2 = T/G_U$. For a single protection scheme, $G_U = PG_A$, so $M_2 = M_1/P$. Mixed schemes are treated by cube in equation (3.13).

M_3 Cost per potentially usable GB including field uplift: $M_3 = M_2(1 + u)$, where u is a fractional uplift.

M_4 Selling price per potentially usable GB at gross margin GM : $M_4 = M_3/(1 - GM)$.

M_5 Selling price per actually usable GB, accounting for average blob and CDF sizes and object-count limits: $M_5 = T(1 + u)/[(1 - GM)G_{U,Actual}]$. The corresponding cost per GB before gross margin is $T(1 + u)/G_{U,Actual}$.

These ratios require positive capacity and $0 \leq GM < 1$. When no user data can be stored, cost per usable GB is undefined.

2 Notation

$\mathbf{G}_R$:= GB of *raw* capacity

$\mathbf{G}_A$:= GB of *available* capacity

$\mathbf{G}_U$:= GB of potentially usable capacity under the protection-only approximation

$\mathbf{C}_R$:= Material cost of *raw* capacity, expressed as $\$:\mathbf{G}_R$

$\mathbf{C}_U$:= Cost with field uplift per actually usable GB

$\mathbf{D} = \{\mathbf{D}_0, \mathbf{D}_1, \mathbf{D}_2\}$ – HDD types contained in Nodes

$\mathbf{D}_0$:= 320GB HDD (Gen 4 HDD)

$\mathbf{D}_1$:= 500GB HDD (Gen 4' HDD)

$\mathbf{D}_2$:= 500GB HDD (Gen 5 HDD)

$\mathbf{N} = \{\mathbf{N}_0, \mathbf{N}_1, \mathbf{N}_2\}$ – Node types, represented by tuples of their installed disks

$\mathbf{N}_0$ = ($\mathbf{D}_0, \mathbf{D}_0, \mathbf{D}_0, \mathbf{D}_0$) (Gen 4 node)

$\mathbf{N}_1$ = ($\mathbf{D}_1, \mathbf{D}_1, \mathbf{D}_1, \mathbf{D}_1$) (Gen 4' node)

$\mathbf{N}_2$ = ($\mathbf{D}_2, \mathbf{D}_2, \mathbf{D}_2, \mathbf{D}_2, \mathbf{D}_2, \mathbf{D}_2$) (Gen 5 node)

$\mathbf{H} = \{\mathbf{H}_0, \mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3\}$ – Fixed overhead, expressed in GB

$\mathbf{H}_0$:= Regeneration buffer (“regen buffer” in the CLI)

$\mathbf{H}_1$:= Database overhead (“system buffer” in the CLI)

$\mathbf{H}_2$:= FilePool + Platform (Linux + Filesystem) overhead (“system resources” in CLI)

$\mathbf{H}_3$:= Audit and metadata overhead

$\mathbf{L}$:= “Cluster object” count limit, anticipated to be 25M:HDD in Hapkin

$\mathbf{Z}$:= Number of HDD in the modeled cluster or homogeneous partition

$\mathbf{P} = \{\mathbf{P}_{\text{cpm}}, \mathbf{P}_{\text{cpp}}\}$ Data protection scheme, over the interval (0,1], where $\mathbf{P}_{\text{cpm}} := \text{CPM}$ and $\mathbf{P}_{\text{cpp}} := \text{CPP}$. Generally, $P_x = m / (m + n)$, where there are m data fragments and n protection fragments.

$\mathbf{R}$:= Titan rack material cost + miscellaneous minor components (cables, etc.), in US\$

$\mathbf{W}$:= G4 Switch material cost (i.e. 2 switches) + 2 PDP + 2 PDU + cables, etc. per cube

$\mathbf{T}$:= Total Solution Cost, expressed in US\$

$\mathbf{A}$:= ASP per actually usable GB

j_i := Number of physical nodes of type i

$\mathbf{u}$:= Fractional field uplift; for example, 10% means $u = 0.10$

A **user object** is the combination of a CDF plus associated blob(s) (typically a user file)

A **cluster object** is a single CDF or blob fragment

$\mathbf{Q}_{\text{obj}}$:= number of user objects in the system partition

$\mathbf{S}_{\text{cdf}}$:= Average CDF size, which can be considered a constant of 4 KiB for our purposes

$\mathbf{S}_{\text{blob}}$:= Average blob size (i.e. the size of the user object being ingested into the cluster)

C'_R denotes uplifted raw cost per GB; A_R denotes raw selling price per GB. $T' = T(1 + u)$ and $T'' = T'/(1 - GM)$ are total uplifted cost and total selling price.

$N = \sum_i j_i$ counts physical nodes. $U = j_0 + j_1 + 2j_2$ is the occupied rack space in 1U units. C is the actual number of populated cubes, with N_c physical nodes and $G_{A,c}$ available GB in cube c .

G'_U accounts for average blob and CDF sizes before object-count limits; $G_{U,Actual}$ also accounts for those limits. Both count logical blob and CDF bytes once, excluding protection copies.

2.1 Notes

1. All costs and prices are in US\$.
2. A physical Gen 4 or Gen 4' node occupies 1U; a physical Gen 5 node occupies 2U. Physical nodes are used for disk counts and protection groups. Rack-space units U are used for enclosure cost, so a 2U node is not counted twice when calculating raw capacity.
3. In earlier versions of the model we factored out the effect of CDF storage and object size to simplify the model. However, we subsequently decided that this simplification masked too much detail. As of v0.50 of the model, we have added consideration for these two factors.
4. Cost of minor elements, e.g. cables, are factored into component cost of switches, nodes and rack(s).
5. We assume new installs.
6. We do not consider the effect of single instance store (SIS).
7. CDFs are written in a $1+n$ protection scheme; i.e. we have $1+n$ copies of each CDF.
8. Capacity units must be consistent throughout a calculation. A GB is 10^9 bytes and a TB is 10^{12} bytes; a GiB is 2^{30} bytes and a TiB is 2^{40} bytes, following the IEC notation in [2]. Thus 500 GB is approximately 465.66 GiB. The numerical value is 6.87% lower when expressed in GiB; the physical capacity is unchanged. Convert blob and CDF sizes into the same units as capacity before applying the object-count equations; for example, 4 KiB is $4096/10^9$ GB. The presentation in [8] uses decimal units for comparison with advertised disk capacities.

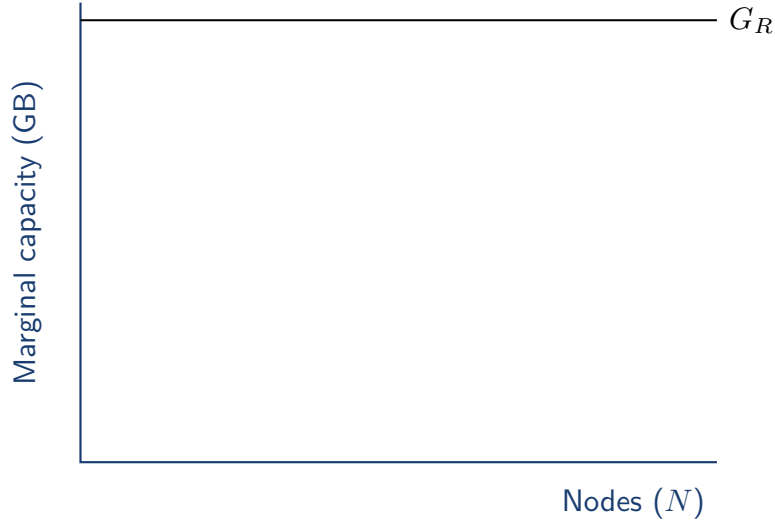
3 Equations

Solution Total Raw Capacity (GB)

$$G_R(N, j) = \sum_{i=0}^2 j_i D_i |N_i| \quad (3.1)$$

Raw capacity is a function of the number and type of nodes in the cluster. We generalize this by taking the cardinality (# of elements) of the node type to determine the appropriate number of HDD; cf. Note 2 (p. 3).

The graph below illustrates this conceptually, where the area under G_R represents **raw** capacity.



Solution Total Material Cost (\$)

$$T = R \lceil U/32 \rceil + WC + \sum_{i=0}^2 j_i c_i \quad (3.2)$$

where c_i is the cost of a physical node of type i

Total material cost includes racks, switches (plus PDP and PDU) and nodes. Each rack provides 32U of modeled node space; each populated cube requires a redundant switch set (W). For tightly packed configurations with 16U per cube, $C = \lceil U/16 \rceil$. Use the actual cube count for partially populated deployments. The rack term assumes tightly packed racks; substitute the actual rack count when it differs.

Material Cost of Raw Capacity (\$:GB)

$$C_R = \frac{T}{G_R} \quad (3.3)$$

Total Material Cost with Field Uplift (\$)

$$T' = T(1 + u) \quad (3.4)$$

EMC has two types of uplift, which we indicate in Figure 1 (p. 13) for completeness. However, only *standard cost* is germane to our calculations, which results in an identical fixed percentage uplift for both Centera and CLARiiON.

Material Cost of Raw Capacity with Field Uplift Applied (\$:GB)

$$C'_R = \frac{T'}{G_R} \quad (3.5)$$

ASP with GM Applied (\$)

$$T'' = \frac{T'}{1 - GM} \quad (3.6)$$

Gross Margin goals vary by product. In [8] we illustrate the effect of varying the GM.

Selling Price per GB of Raw Capacity (\$:GB)

$$A_R = \frac{T''}{G_R} \quad (3.7)$$

Total Fixed Overhead (GB)

Regen Buffer:

The characteristics of the regen buffer are enumerated below.

- There is one regen buffer allocated for each Cube on the cluster.
- The Sysop reserves the regeneration buffer via the CLI command: set capacity regeneration buffer. The Sysop selects what type of protection is desired by inputting the number of HDD to protect. To affect node-level protection, the Sysop would enter 4 HDD for a Gen4 (S1) node and 6 HDD for a Gen5 (S2) node.
- The size of the regen buffer is determined based on a cluster scope. That is, there are not different sized regen buffers per Cube in the cluster. Conceptually, if you had a cluster occupying 128U with 126 nodes of S1 and 1 node of S2, to provide node-level protection would require that the Sysop set the number of HDD to 6. This value would then be used for all 8 Cubes in our hypothetical cluster.
- Nodes in a cluster are intended to use the regen buffer within their Cube. However, they will use the regen buffer of a different Cube (HRG), if necessary.
- The total regen buffer allocation for the cluster is the sum of the individual regen buffers per Cube/HRG.

Let $D_{\max} = \max_{i:j_i > 0} D_i$ be the largest installed disk capacity, and let h_c be the number of disks reserved for regeneration in cube c . The reservation and mirror-group factor MG are configuration inputs, with $MG \in \{1, 2\}$.

$$H_0 = \sum_{c=1}^C R_{buf,c}, \quad R_{buf,c} = h_c D_{\max} MG \quad (3.8)$$

For a cluster-wide setting, $h_c = h$ for every cube. At $MG = 2$, a four-disk reservation on 500 GB disks is 4000 GB per cube; a six-disk reservation is 6000 GB per cube. The model uses the

configured reservation rather than selecting a disk or node type by testing whether its nominal capacity is zero.¹

The historical description gives both a uniform cluster-wide setting and a different rule for cubes below eight nodes. Those operational rules are not mutually resolved by the source. The appropriate h_c and MG must therefore be confirmed for the release and configuration being modeled.

Database Overhead / Audit and Metadata Overhead:

$$H_1 = k_1 N, \quad H_3 = k_3 N \quad (3.9)$$

System Resources:

$$H_2 = k_2 \sum_{i=0}^2 j_i |N_i| \quad (3.10)$$

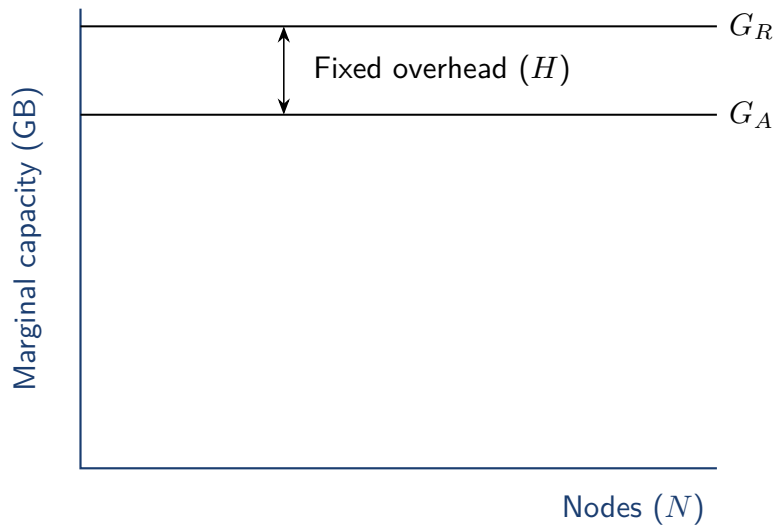
Here k_1 and k_3 are release-specific reservations in GB per physical node, and k_2 is the system-resources reservation in GB per HDD. They are distinct constants.

Effective Total Available Capacity (GB)

$$G_A = G_R - \sum_{i=0}^3 H_i \quad (3.11)$$

We treat database overhead as a fixed constant by release. Therefore, available capacity for user data storage is simply raw capacity minus this overhead. The total fixed overhead is $\sum_i H_i$. Only configurations with nonnegative available capacity are feasible.

The graph below illustrates this conceptually, where the area under G_A represents effective **available** capacity.



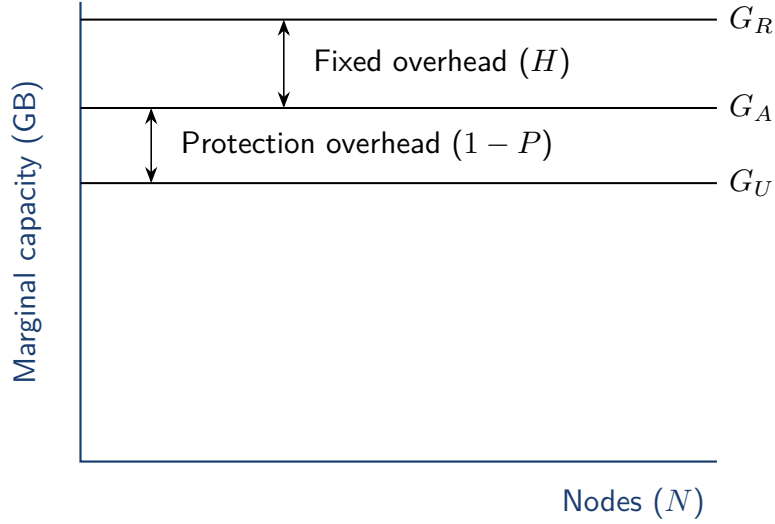
¹The original document attributes the factor of 2 to the first release's dual power rails and cites [5]. It describes reducing that factor to 1 as a goal for Hapkin.

Effective Usable Capacity: Single Protection Scheme (GB)

$$G_U = G_A \frac{m}{m+n} \quad (3.12)$$

Equation (3.12) provides usable capacity when a single protection scheme (P , where $P = m/(m+n)$) is employed cluster-wide. It does not take into account object limits or the different protection applied to CDFs. It is exact when all logical bytes use the same protection ratio, and an approximation when CDFs are small relative to blobs.

The graph below illustrates this conceptually, where the area under G_U represents effective **usable** capacity.



Usable capacity is principally a measure of storage efficiency vis-à-vis raw capacity. We consider three elements as material with respect to storage efficiency:

- Data protection scheme P ,
- Overhead $\{H\}$, and
- The limiting effect of a fixed cluster object count/HDD.

With Centera, customers can choose one of two **data protection schemes**: mirroring (P_{cpm}) or parity (P_{cpp}). For P_{cpm} , $m = 1$ and $n = 1$; for P_{cpp} , $m = 6$ and $n = 1$. Regardless of which data protection scheme is chosen, CDFs are always stored as P_{cpm} . We generalize the equations to handle any $m+n$ protection scheme.

Overhead for Centera consists of the four elements described in §2 for $\{H\}$. We treat *database overhead* as a constant since DB overhead is fixed by filesystem volume reservation. That space is thus unavailable for customer data irrespective of the actual size of the database².

We are likewise able to treat the combination of C* (FilePool) and Platform (Linux+Filesystem) overhead as a constant by release. Here too we will use the expected values for Grimbergen and will adjust as necessary for Hapkin.

²Note that while this space is a constant for a given release of C*, this amount of space reserved for the C* databases historically has increased with each major revision. For our calculations we will use the expected reservation amount for Grimbergen.

The maximum number of cluster objects per HDD acts as a limit L . For Hapkin the anticipated max fragment count per HDD is 25M. While this is a consideration for Grimbergen and its ancestors³, it is not anticipated to be an issue in Hapkin due to the introduction of Containers.

Max Potential Usable Capacity: Combined Protection Schemes (GB)

Choosing the CPP protection scheme complicates the equation because we cannot be guaranteed that *all* objects will in fact be stored with CPP. As a result of this asymmetry we cannot treat P as a single value because it is possible that there will be two data protection schemes employed within the same cube.

P takes on a single, constant value when using CPM across the *entire* cube (therefore, $P = P_{cpm}$), or – when using CPP⁴ – if the number of *available* nodes is a multiple of 8 per HRG (heuristic reliability group), in which case $P = P_{cpp}$.

A more complex equation comes in to play when CPP is the *desired* protection scheme, but it cannot actually be employed by FilePool because the number of *available* nodes is not a multiple of 8. Equation (3.13) provides effective usable capacity in such a case (without taking into account the effect of object limit).

The usable capacity across the entire cluster ($G_{U,Cluster}$) is the sum of the usable capacity for the individual Cubes that make up the cluster ($G_{U,Cube}$).

$$f_{c,cpp} = \frac{8\lfloor N_c/8 \rfloor}{N_c}, \quad f_{c,cpm} = 1 - f_{c,cpp}, \quad (3.13)$$

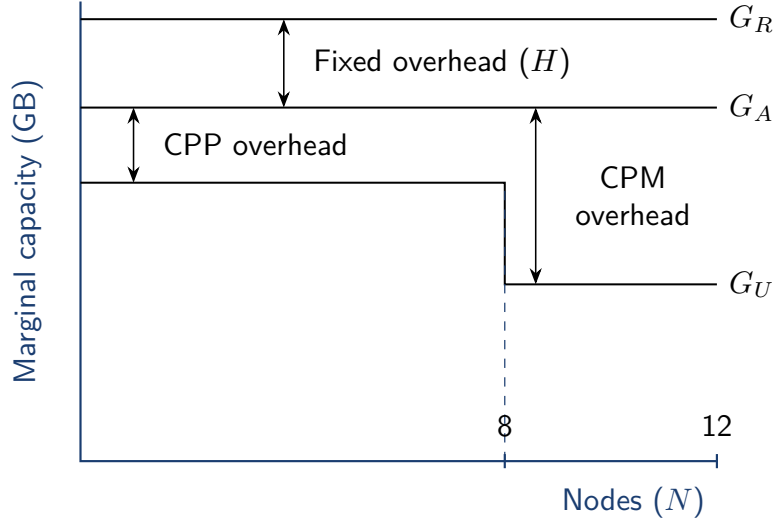
$$G_U = \sum_{c=1}^C G_{A,c}(f_{c,cpp}P_{cpp} + f_{c,cpm}P_{cpm}).$$

For populated cubes, $N_c > 0$. The fractions in (3.13) assume equal available capacity per physical node within each cube. For unequal nodes, use capacity fractions calculated from the actual allocation instead. When CPM alone is selected, set $f_{c,cpp} = 0$ and $f_{c,cpm} = 1$. In (3.13), for each Cube we apportion available storage capacity over the two protection schemes (P_{cpp} and P_{cpm}). We then sum the usable capacity of the Cubes within a single logical cluster to get usable capacity cluster wide. Of course, (3.13) is just a generalized form of (3.12). Finally, the reader should note that equation (3.13) is an optimization algorithm that biases to P_{cpp} . In practice, C^* will tend to be less efficient as C^* doesn't operate in the idealized environment of a model (this is discussed in greater detail in §4.1, p. 12).

The graph below illustrates the case where an 8 node cluster was subsequently upgraded to 12 nodes. The first 8 nodes will have CPP protection overhead (P_{cpp}), while the 4 nodes added subsequent to the 8 nodes (which presumably are nearly full, leading to the additional nodes being added to the cluster) will have CPM protection overhead (P_{cpm}). The area under G_U represents usable capacity.

³Cf. [4] for an illustration of the limiting effect of object (fragment) count on storage in Faro.

⁴Readers are directed to [1] for detail about CPP in Grimbergen and Hapkin.



Our expectation is that with the introduction of Containers in Hapkin the limiting effect of object count is removed ($L = \infty$). We show that model first (M_2 in the conceptual descriptions provided on p. 1). Later we add the effect of object limitation into the model to illustrate the result if object count remains a limiting factor.

Usable Capacity with Effects of Object & Capacity Limits

The next level of detail to consider is the two limitations in our system. The first is the number of cluster objects per HDD, which is limited to L . This limitation is required to impose a lower bound on MTDDL (Mean Time to Data Loss) as it impacts regeneration speed. The second limitation is the total amount of available capacity G_A . We'll describe each of these limitations in turn, but we begin by describing a single **user** object write in terms of number of **cluster** objects and capacity usage.

For a homogeneous protection partition, writing a single user object results in $m + n$ blob fragments and $1 + n$ CDF copies. For the CPM and CPP schemes considered here, $n = 1$, so the CDF is mirrored in both cases. This means that in total, a single user object will result in $m + n + 1 + n = m + 2n + 1$ cluster objects.

The inequality below specifies the effect of cluster object limit.

$$\begin{aligned}
 (m + 2n + 1) \cdot Q_{obj,L} &\leq L \cdot Z \\
 \text{where } L &= \text{a release dependent constant} \\
 \text{and } Z &= \sum_{i=0}^2 j_i |N_i|
 \end{aligned} \tag{3.14}$$

The object-budget upper bound on the number of user objects in a homogeneous partition is:

$$Q_{obj,L} = \frac{L \cdot Z}{m + 2n + 1} \tag{3.15}$$

Writing a single user object results in $\frac{m+n}{m} S_{blob} + (1 + n) S_{cdf}$ capacity usage.

The limitation due to available capacity is:

$$\left(\frac{m+n}{m}S_{blob} + (1+n)S_{cdf}\right) \cdot Q_{obj,cap} \leq G_A \quad (3.16)$$

The capacity upper bound on the number of objects in a homogeneous partition is:

$$Q_{obj,cap} = \frac{G_A}{\frac{m+n}{m}S_{blob} + (1+n)S_{cdf}} \quad (3.17)$$

As in (3.13), we must account for the fact that there may be differing protection schemes across the cluster. Define b_s as physical capacity consumed per logical user object under scheme s . The cluster-wide capacity upper bound is:

$$\begin{aligned} b_s &= \frac{m_s + n_s}{m_s} S_{blob} + (1 + n_s) S_{cdf}, \\ q_{cap,c,s} &= \frac{G_{A,c} f_{c,s}}{b_s}, \quad Q_{obj,cap} = \sum_{c=1}^C \sum_{s \in \{cpp, cpm\}} q_{cap,c,s}. \end{aligned} \quad (3.18)$$

Here $(m_{cpp}, n_{cpp}) = (6, 1)$ and $(m_{cpm}, n_{cpm}) = (1, 1)$. Object sizes must use the same capacity units as $G_{A,c}$; object counts are continuous upper bounds and should be rounded down within each partition when discrete counts are needed. In (3.13) we determine the max potential usable capacity. In (3.18) we get more granular by determining what the actual maximum usable capacity is for the specific average blob size (S_{blob}).

$$G'_U = Q_{obj,cap} \times (S_{blob} + S_{cdf}) \quad (3.19)$$

As S_{cdf}/S_{blob} tends to zero, G'_U approaches the protection-only capacity G_U ; equality is not guaranteed for finite CDF sizes. Large blobs also avoid the small-object CPP threshold, although the placement constraints in §4.1 still apply.

Effect of Object Limit

For a homogeneous partition, the number of stored user objects is bounded by $\min(Q_{obj,cap}, Q_{obj,L})$. For mixed protection, limits must be applied within each partition before summing; a cluster-wide minimum can hide a local object-budget bottleneck.

Let $Z_{c,s}$ be the HDD-equivalent object budget allocated to partition (c, s) , with $\sum_s Z_{c,s} \leq Z_c$, where Z_c is the number of HDDs in cube c . Define

$$q_{L,c,s} = \frac{L Z_{c,s}}{m_s + 2n_s + 1}.$$

The allocation is a model input. It assumes fragments can be balanced within the allocated budget; the aggregate bound alone does not guarantee feasible placement on every physical disk. The reduction in logical usable capacity attributable to object limits is

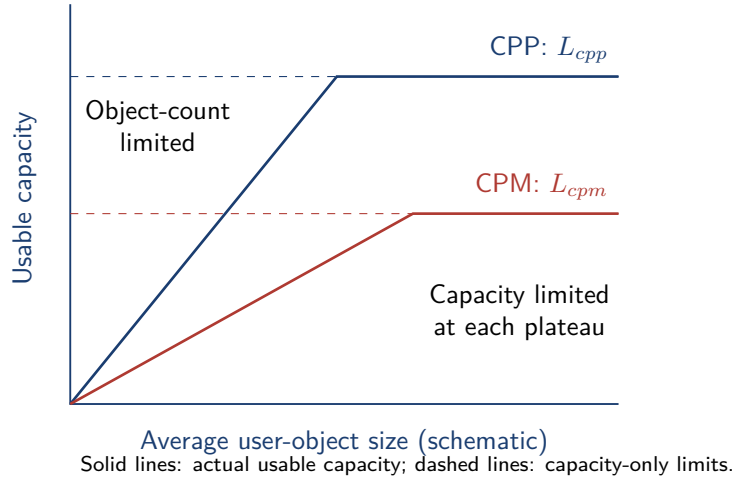
$$G_{unused} = \sum_{c,s} \max(0, q_{cap,c,s} - q_{L,c,s})(S_{blob} + S_{cdf}). \quad (3.20)$$

Here G_{unused} denotes lost logical capacity, not unoccupied raw disk space. Actual logical usable capacity is therefore

$$\begin{aligned} G_{U,Actual} &= G'_U - G_{unused} \\ &= \sum_{c,s} \min(q_{cap,c,s}, q_{L,c,s})(S_{blob} + S_{cdf}). \end{aligned} \quad (3.21)$$

With no object bottleneck, $G_{U,Actual} = G'_U$. If a partition's object limit binds, only that partition's usable capacity is reduced. The same formulas apply to a single protection scheme by using one partition.

The graph below conceptually illustrates the effect that object limitation has with respect to the two protection schemes. The curves L_{cpp} and L_{cpm} give achievable usable capacity at each object size. Each scheme has its own capacity-only plateau. The axes are schematic: no measured object-size thresholds or release-specific capacities are implied.



Cost and ASP per Actually Usable GB

The cost-over-capacity construction used in equations (3.3), (3.5) and (3.7) gives uplifted cost and selling price per actually usable GB:

$$C_U = \frac{T'}{G_{U,Actual}}, \quad (3.22)$$

$$A = \frac{C_U}{1 - GM}. \quad (3.23)$$

These ratios require $G_{U,Actual} > 0$ and $0 \leq GM < 1$. The gross-margin target is specific to the product line.

4 Model Considerations

Axiom #1: With C*, things are seldom black or white.

In our model we make a number of simplifying assumptions to keep the model from becoming unreadable due to all the conditional branching that would be required if we chose an exact match to C* behavior. In this section, we describe some of those differences to ease the mind of the purist.

4.1 Multiple Protection Schemes

As stated previously, the algorithm depicted in equation (3.13) is an optimization that biases to CPP. In practice, one cannot predict a priori in all cases whether C* will actually store an object with CPP protection. Rather, this determination is made by C* on a per-write basis. This is especially true when capacity, as a unit measure of node type, is asymmetric. Specifically, when a cluster has a combination of N_1 and N_2 nodes, there's an intrinsic imbalance in capacity:node. Since the CPP protection scheme has a node scope (as opposed to a disk scope), this asymmetry will result in some objects written in CPM – even if CPP is the Sysop selected protection scheme.

To give the reader a sense of the number of conditionals involved when deciding when to write via CPP versus CPM, we provide pseudo-code below (biasing to CPP). Of course, the pseudo-code is a simplification of the actual algorithm.

```
CPP_THRESHOLD = 100000 // bytes; historical model input
CPP = {data: 6, parity: 1, min_nodes: 8}
CPM = {data: 1, parity: 1, min_nodes: 2}

function WriteObject(object):
    for scheme in [CPP, CPM]:
        if CheckWrite(scheme, object):
            return write_blob_and_mirrored_cdf(scheme, object)
    return failure

function CheckWrite(scheme, object):
    if scheme == CPP and size(object) < CPP_THRESHOLD:
        return false
    if nodes_available < scheme.min_nodes:
        return false
    fragment_size = ceil(size(object) / scheme.data)
    return can_place_fragments_and_mirrored_cdf(
        scheme, object, fragment_size)
```

The placement predicate includes per-node free capacity, object slots, and protection-group constraints for both the blob and its CDF. These release-dependent checks must be supplied by the implementation; aggregate free capacity is insufficient. The CPP size threshold applies only to CPP, so a small object may fall back to CPM. The eight-node CPP eligibility rule is the assumption used in the preceding model, rather than the count of seven fragments.

4.2 Database Overhead

We only consider “hard overhead” in our model; i.e., the overhead reservations that come with a given C* release. We do not consider “soft overhead.” Readers interested in gaining a greater understanding of this are directed to [3].

5 EMC Cost Categorization

EMC has two assigned cost categories, referred to as *standard* and *product*. These two categories are further divided into *software* and *hardware*. For both Centera and CLARiiON, *standard cost* results in a **10% uplift** to (hardware) material cost; there is no uplift assigned to the C* software.

Standard margin is reported against *standard cost* at the division level. This is the GM (gross margin) figure reported at the General Manager level and the one we are concerned about in our model. *Product cost* is an accounting number presented to field Sales, principally as a means of setting a discount floor. It differs – somewhat significantly – by product. However, we do not have to consider *product cost* in our calculations.

Figure 1 below illustrates this cost categorization.

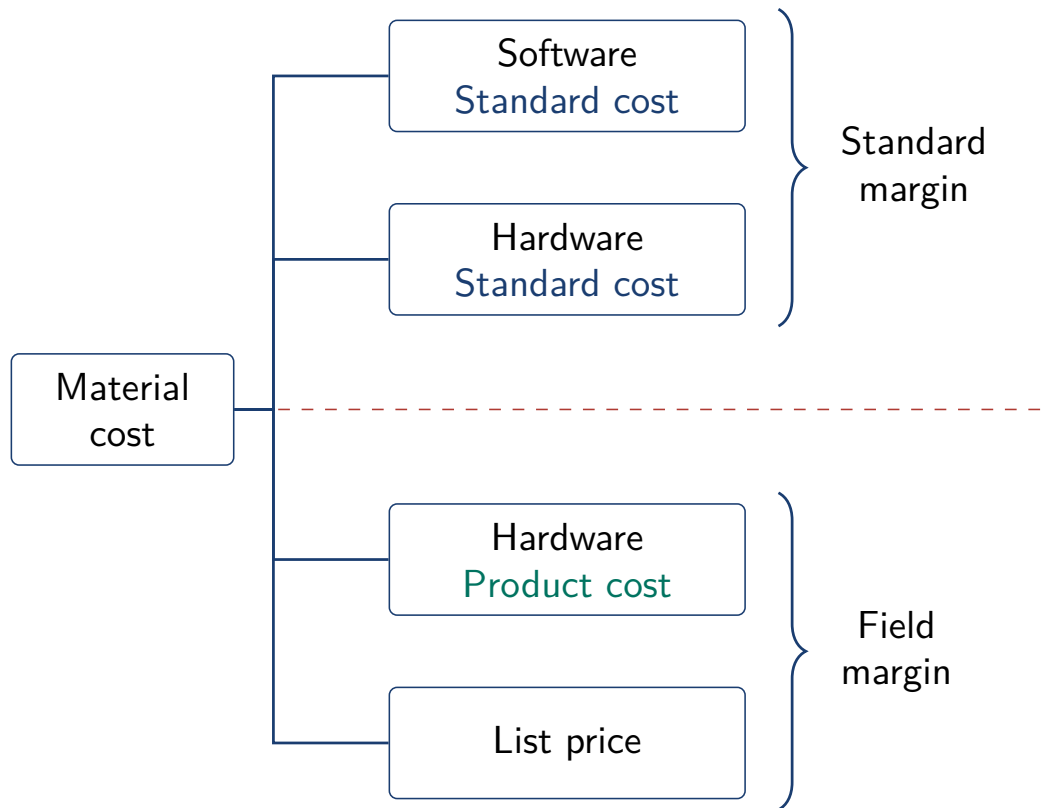


Figure 1: EMC Cost Categorization

6 References

- [1] C. D’Halluin. *Functional Specification – CPP 6+1 on 8-node Systems*, v0.1, June 20, 2005.
- [2] IEC 60027-2. *Letter symbols to be used in electrical technology - Part 2: Telecommunications and electronics*, November 11, 2000.
- [3] F. Matthijs. *Suicide Prevention for New Disk Layout, DB/disk*, v0.8, February 16, 2005.
- [4] R.Primmer et al. *Storage Cost BCAD*, v1.00, 27 April 2005, pp. 19-20.
- [5] Ibid. pp. 20-21.
- [6] Ibid. p. 19.
- [7] Ibid. p. 18.
- [8] R.Primmer. *Storage Cost – Excel Implementation*, v0.10, 16 September 2005.

All references available at: <http://csgwiki.dyndns.org/bin/view/PM/SafariTwo>

7 Acknowledgements

We would like to acknowledge the following people for their help:

- **Bart Vanhoutte** and **Frank Matthijs** for their help in understanding the particulars of how the regen buffer is allocated across all cubes of the cluster;
- **Scott Nyman** for his help in working through the “first brick on a new cube” problem in the Excel model;
- **Vai Sharma** for his help in understanding the various EMC models for field uplift and for detail on CLARiiON ASP and GM; and
- **Frank Matthijs** and **Kristof Van Belleghem** for their help in understanding how CentraStar operates on soft reservations for database volumes, and the anticipated usage in Grimbergen.

We would also like to acknowledge the following reviewers for their comments to early drafts of this paper. Specifically: **Brian Mahan**, **Scott Nyman**, **Jiri Schindler** and **Bart Vanhoutte**.